

Insertion Sort with Duplicate Elements

Aim

To implement the **Insertion Sort algorithm in Python** for an array containing **duplicate elements** and sort the elements in ascending order.

Algorithm

1. Start with the second element of the array.
2. Consider the current element as the `key`.
3. Compare the key with elements in the sorted portion.
4. Shift elements greater than the key one position to the right.
5. Insert the key into its correct position.
6. Continue until all elements are processed.
7. Display the sorted array.

`\text{Learn more}`

Procedure

1. Open a Python compiler or IDE.
2. Create an array containing duplicate elements.
3. Define an `insertion_sort()` function.
4. Start from the second element.
5. Store the current element in a variable called `key`.
6. Compare the key with the elements before it.
7. Shift larger elements to the right.
8. Insert the key in its correct position.
9. Repeat until the entire array is sorted.
10. Display the original and sorted arrays.

Python Code

```
def insertion_sort(arr):  
    for i in range(1, len(arr)):  
        key = arr[i]  
        j = i - 1  
        while j >= 0 and arr[j] > key:  
            arr[j + 1] = arr[j]  
            j = j - 1
```

```
        arr[j + 1] = key

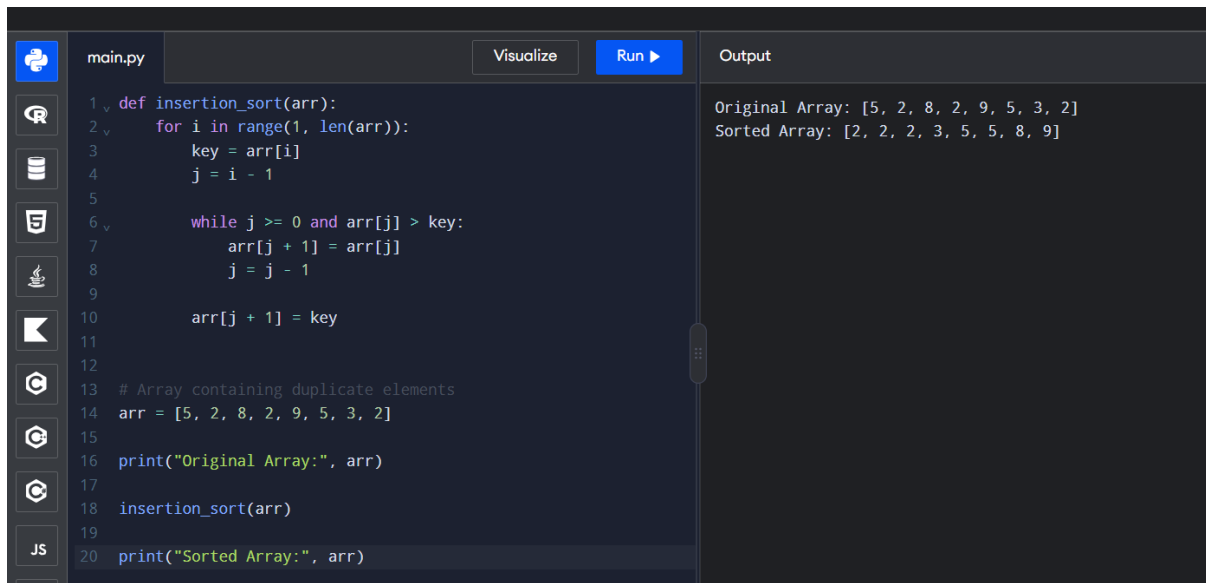
# Array containing duplicate elements
arr = [5, 2, 8, 2, 9, 5, 3, 2]

print("Original Array:", arr)

insertion_sort(arr)

print("Sorted Array:", arr)
```

Output



The screenshot shows a code editor with a file named 'main.py'. The code implements the Insertion Sort algorithm. It defines a function 'insertion_sort(arr)' that iterates through the array, taking each element as a 'key' and inserting it into its correct position by shifting larger elements to the right. The main code creates an array 'arr' with duplicate elements [5, 2, 8, 2, 9, 5, 3, 2], prints it, calls 'insertion_sort(arr)', and prints the sorted array. The output panel on the right shows the original array and the sorted array [2, 2, 2, 3, 5, 5, 8, 9].

```
1 def insertion_sort(arr):
2     for i in range(1, len(arr)):
3         key = arr[i]
4         j = i - 1
5
6         while j >= 0 and arr[j] > key:
7             arr[j + 1] = arr[j]
8             j = j - 1
9
10        arr[j + 1] = key
11
12
13 # Array containing duplicate elements
14 arr = [5, 2, 8, 2, 9, 5, 3, 2]
15
16 print("Original Array:", arr)
17
18 insertion_sort(arr)
19
20 print("Sorted Array:", arr)
```

Output

```
Original Array: [5, 2, 8, 2, 9, 5, 3, 2]
Sorted Array: [2, 2, 2, 3, 5, 5, 8, 9]
```

Result

The Insertion Sort algorithm was successfully implemented in Python. The array containing duplicate elements was correctly sorted in ascending order, while preserving all duplicate values. The time complexity of Insertion Sort is **$O(n^2)$** in the average and worst cases and **$O(n)$** in the best case when the array is already sorted.